LIDR

Módulo 2 · Los prompts de la sesión

Todo lo que se teclea hoy, en orden, para que puedas seguir la sesión sin copiar del chat y para que puedas repetirla después en tu propio proyecto, que es donde de verdad sirve.

No es el guion del mentor. Aquí está lo que se ejecuta y **por qué cada restricción está donde está**, que es la parte que se pierde cuando alguien copia un prompt suelto.

Hoy **no se escribe código de producto**. Lo que sale de la sesión son documentos: un PRD y un backlog, versionados en el repositorio al lado del código. El arco es entero, de una idea difusa a tickets ejecutables.

Antes de empezar

Trabaja sobre **tu propio fork** de:

`https://github.com/LIDR-academy/flowsync-ai4devs`

Fork, no clone directo: el harness que montaste en el Módulo 1 lleva escrito en su `CLAUDE.md` que antes de tocar código se crea una rama y que al cerrar la tarea se abre un PR. Hoy no se toca código —se escriben documentos— pero **hoy se commitea tres veces**, y el agente puede generalizar esa regla y montarte el ciclo entero. Si lo hace sobre un clon del repositorio del curso, **no tienes permiso de push** y el paso te revienta a mitad. Sobre tu fork, no. La secuencia es **fork → clone de tu fork →** `git checkout s2/start` → `make setup` (o `make start` a secas si ya hiciste el setup en el Módulo 1).

La rama de partida es `s2/start`, que es el resultado del Módulo 1: trae el harness completo (`CLAUDE.md` / `AGENTS.md`, las skills `commit` y `priority-ticket`, el subagente `adversarial-reviewer`, el hook de Prettier y el `.mcp.json` de Atlassian) y el login del frontend. Lo que **no** trae todavía es la carpeta `docs/`: el PRD y el backlog nacen hoy, en pantalla.

> 🧭 `Shift+Tab` **cicla los modos, no es un interruptor.** La primera mitad de la sesión se hace en **plan mode** (la IA explora y pregunta sin escribir nada), y hay que **salir** antes del primer prompt que escribe a disco. Púlsalo **hasta que la barra de estado diga** `plan mode`: si te pasas, aterrizas en auto-accept y el agente escribe sin pedirte permiso.

> 🔌 **Sobre el tablero de Jira.** El último bloque de la sesión sube las historias y sus tickets a un tablero de Jira por MCP. En el directo ese tablero es el del curso (`FLOW`), así que **lo verás correr en la pantalla del mentor**. Para reproducirlo por tu cuenta necesitas un tablero de Atlassian propio, y hay tres cosas del prompt que dependen de cómo esté montado el tuyo: que el proyecto tenga la clave `FLOW`, que los tipos de incidencia se llamen `Historia` y `Subtarea` (si tu sitio está en inglés serán `Story` y `Sub-task`, y hay que sustituirlos en el prompt) y que las **subtareas estén habilitadas**. Todo lo demás de esta hoja funciona sin Jira.

> 🖥️ **Dónde se teclea qué.** Los prompts van dentro de `claude`. Las líneas que empiezan por `!` también: el `!` ejecuta el comando en el shell de Claude Code **sin pasar por el agente**, y eso importa, porque un clon limpio no trae permisos preconfigurados y si el agente lo lanzara como herramienta te pediría permiso.

Mapear el terreno antes de especificar

Con **plan mode** activo. Tu proyecto real casi nunca es una carpeta vacía, y FlowSync tampoco: ya trae `auth` del módulo anterior.

```
Antes de nada, léete el repo y devuélveme el estado real del proyecto: qué capabilities ya existen (auth…) y un diagrama rápido del modelo de datos actual (entidades y relaciones, en Mermaid). No propongas cambios todavía; solo quiero el mapa de lo que hay.
```

"No propongas cambios todavía" es lo que separa esto de una sesión de brainstorming. Lo único que quieres ahora es el mapa, y si le dejas proponer, el mapa llega mezclado con la propuesta y ya no sabes qué había y qué se inventó.

⚠️ **Este mapa es para entender, no es la verdad absoluta.** La IA lee el código y a veces se equivoca. Y fíjate en que no le pides que convierta *todo* el sistema en specs de golpe: eso es un artefacto derivado que envejece mal. Mapeas lo justo para aterrizar lo que vas a construir.

> 💡 **El diagrama del modelo de datos NO va a entrar en el PRD.** Sirve para que la IA y tú entendáis el terreno, y nada más. En cuanto veas tablas, endpoints o nombres de clase dentro de un PRD, alguien bajó de nivel antes de tiempo.

Hacer que la IA pregunte en vez de proponer

El pitch del fundador va **dentro** del prompt, para que no dependa de que alguien lo teclee bien en vivo.

```
Actúa como product partner escéptico. Aquí tienes una idea difusa para FlowSync: «Quiero que FlowSync sea una herramienta para que los equipos remotos sepan en qué está trabajando cada uno sin tener que hacer reuniones de sincronización. Algo tipo tareas compartidas pero más en tiempo real y menos rollo que Jira.» Antes de proponer nada, hazme las 5 preguntas que más reducirían la incertidumbre sobre el problema, los usuarios y el alcance. No asumas; pregunta. Ten en cuenta el estado del repo que acabas de mapear (auth; las tareas aún no existen). Una sola ronda: con mis respuestas, continúa sin abrir otra tanda de preguntas. Quédate a nivel de producto: nada de modelo de datos, esquema, estados internos, endpoints ni latencias, eso se decide más adelante.
```

Tres restricciones, y las tres hacen falta:

"Hazme las 5 preguntas… No asumas; pregunta" es lo que convierte a la IA en socia de pensamiento en vez de en generadora de listas de features. Lo que **no** se le dice es *qué* preguntar: eso lo elige ella, y ahí está el valor.

"Una sola ronda" es puro control de alcance. Sin esa línea abre tres tandas más de preguntas y la conversación no termina nunca.

"Quédate a nivel de producto" es el guardarraíl de toda la sesión. Sin él baja al modelo de datos, a los estados internos y a las latencias, y ya no estás decidiendo qué construir sino cómo.

> ⚠️ **Acotar al agente no es hacer trampa.** Esto no es determinista: si lo dejas suelto, cada ejecución toma un camino distinto. Poner límites es exactamente lo que vas a hacer en cuanto tengas una tarea real y un plazo.

Las respuestas del fundador, decididas de antemano

La IA hará unas cinco preguntas y **su redacción cambia en cada ejecución**, pero la sustancia se repite: a quién le duele, qué pasa hoy, qué es "tiempo real", quién mantiene el estado, si sustituye o convive, y cómo se mide el éxito. Por eso la respuesta va escrita antes y se pega **entera, de una vez**, en vez de improvisar producto delante de una clase.

- Qué duele hoy: la daily de sincronización y el "¿en qué estás?" constante por Slack/chat. Nadie ve el estado del equipo sin interrumpir a alguien.
- Quién cobra el valor: los pares, no un lead. No hay reporte hacia arriba y a un manager le daría igual. Duele a los dos devs que descubren tarde que iban a lo mismo, y al que interrumpe a otro para preguntar.
- Episodio concreto: dos personas del equipo tocaron el mismo módulo la misma semana porque una empezó sin que la otra lo supiera. Dos días perdidos.
- Qué reunión desaparece (respuesta honesta, no la vendas de más): la daily NO desaparece entera. Desaparece la ronda de "¿en qué estás?", que hoy se come la mitad de los 15 minutos. La parte de bloqueos sigue, y este MVP no la resuelve.
- Usuarios / equipo: equipos remotos pequeños, 3–10 personas. Roles planos: en el MVP todos ven y editan lo mismo, sin jerarquía de permisos.
- Primer usuario concreto: equipo de 6 personas de producto SaaS, en 3 husos horarios, que hoy usa un gestor de tareas pesado y una daily de 15 minutos por videollamada. Es un CASO DE ESTUDIO, no un cliente real: dilo en voz alta en vez de fingir lo contrario.
- Fronteras: un espacio único compartido, sin entidad "equipo". Varios equipos separados, o gente en más de uno, queda FUERA del MVP: se anota como supuesto en el PRD, no se construye.
- "Tiempo real" = ver los cambios de estado de las tareas sin refrescar ni preguntar. NO es chat, NO es videollamada, NO es colaboración simultánea sobre el mismo documento.
- Es frescura, no presencia: el estado es de la TAREA, no de la persona. Nada de "quién está conectado ahora" ni indicadores de actividad; eso es vigilancia y lo rechazamos a propósito.
- Forma de la señal: resumen que espera, no aviso que interrumpe. El caso es "llego por la mañana o vuelvo de una reunión y veo qué se ha movido". Sin notificaciones push.
- Qué decisión cambia: no empezar algo que otra persona ya está tocando, y elegir lo siguiente sabiendo qué está libre. Si la única respuesta fuera "sentirse informado", el tiempo real no valdría lo que cuesta.
- De dónde sale el estado: lo teclea la persona que hace la tarea, en segundos. Derivarlo de señales externas (Git/PRs, CI, calendario) está FUERA del MVP: es otro producto, con integraciones y OAuth de terceros.
- Por qué se sostiene: no porque sea más agradable, sino porque son dos clics sobre una lista ya abierta, sin campos obligatorios, sin decidir sprint ni estimación. Y quien lo escribe cobra en el momento: esa misma lista es su cola de trabajo, la mira para decidir qué coge, y de paso deja de recibir interrupciones preguntándole cómo va. Si el beneficio fuera solo para los demás, no lo escribiría.
- Si la información se queda vieja: el producto pierde el sentido, y lo asumo. Es el riesgo #1 a validar, no un detalle. La mitigación es que actualizar cueste dos clics, no obligar a nadie.
- Es donde se hace el trabajo, no donde se cuenta: sustituye al gestor de tareas, no convive con él. FlowSync crea las tareas, no lee las de otro sitio. Convivir exigiría doble actualización, que es como muere esta categoría.
- Renuncia explícita a sprints, estimaciones, épicas, backlog priorizado e informes. Un equipo que necesite eso no es nuestro usuario.

```
- "Menos rollo que Jira" = crear una tarea y cambiarle el estado en segundos, sin flujos de configuración ni campos obligatorios. Lo mínimo para saber quién está en qué.
- Qué necesita una tarea en el MVP: título, responsable, estado y fecha de vencimiento. La fecha, para ver de un vistazo qué se ha pasado de plazo.
- Cómo se consume la lista: filtrando por estado, para centrarse en lo pendiente.
- Éxito para el usuario: dejar de hacer la ronda de "¿en qué estás?" de la daily porque el estado del equipo se ve de un vistazo.
- Criterio a una semana de uso real: que el equipo cancele esa ronda y nadie pida que vuelva. Si la siguen haciendo igual, no funcionó.
- Cuánto construir: una vertical fina y usable de punta a punta, no el andamiaje amplio de un producto. Prefiero una capability terminada a tres a medias.
```

> 📄 **En el PDF esta ficha ocupa más de una página, y eso tiene una consecuencia al copiarla:** al seleccionarla de punta a punta se cuela en medio la línea del encabezado y el pie (`AI4DEVS · BLOQUE FUNDACIONAL … / MÓDULO 2 … AI4DEVS · LIDR`). Bórrala antes de pegar. Es lo único del bloque que no forma parte de la respuesta.

⚠️ **No la recortes "porque no me lo ha preguntado".** Dos líneas son obligatorias aunque la IA no pregunte por ellas: "*Qué necesita una tarea en el MVP*" y "*Cómo se consume la lista*". En las pruebas de preparación **nunca** las preguntó, y de ellas dependen las dos historias que se enriquecen al final de la sesión. Si no se siembran aquí, el PRD no las recoge y el fallo aparece tres bloques más tarde, cuando ya no sabes de dónde vino.

Si empuja hacia features que no quieres, el comodín:

```
- Fuera del MVP: notificaciones push, integración con Slack, roles/permisos avanzados, analítica/reporting, comentarios en tareas.
```

Y si aun así baja al modelo de datos —pese a que el prompt anterior se lo prohíbe—, la red de contingencia. Que haga falta es **dato**: significa que el guardarraíl no aguantó.

```
- Estados: tres, fijos y no configurables: pendiente / en curso / hecho. "En curso" es la señal que el producto existe para transmitir, así que está DENTRO. "Bloqueado" es otra cosa y está FUERA: es la mitad de la daily que declaramos no resuelta. Los nombres exactos del enum en código bajan a la spec de implementación, no al PRD.
- Obligatoriedad: responsable SÍ (una tarea sin responsable no cumple el propósito); fecha de vencimiento NO (opcional, y sin fecha nunca aparece como vencida). "Sin campos obligatorios" iba contra la configuración tipo Jira, no contra estos dos.
- Latencia: 5–10 segundos vale. El sync en tiempo real de E3 no entra en este MVP.
- Tests: sí, el trabajo llega con tests. Darlo por hecho en el alcance.
```

> 🔍 **Lee los supuestos que declara al cerrar.** La IA termina diciendo qué rellenó por su cuenta. **Corrige solo los que estén mal.** Ahí es donde se cazan los huecos que la ficha no cubría, no añadiendo más respuestas.

Que la IA proponga alcance, y recortarlo

```
Con mis respuestas, propón el alcance de un MVP: problema, usuarios, propuesta de valor, alcance (in) y NO-alcance (out). Sé agresivo recortando: es un MVP, no el producto final. Justifica cada exclusión.
```

"Sé agresivo recortando" y **"justifica cada exclusión"** trabajan juntas: la primera empuja a quitar, la segunda impide que quitar sea gratis. Un NO-alcance sin argumento no se puede defender ante nadie.

El momento que importa es cuando propone de más —notificaciones push, integración con Slack, roles y permisos— y tú lo mandas fuera. **El NO-alcance es la decisión más valiosa de un MVP**, porque es la única que protege la hipótesis central de todo lo demás.

> ⚠️ **Prepárate para que te discuta bien y para sostener la decisión igual.** En las pruebas argumentó, con buen criterio, que la fecha de vencimiento tira hacia el mismo sitio que los sprints y las estimaciones que el fundador acababa de descartar. Tiene parte de razón. Y aun así se mantiene, porque es lo que se pidió ver de un vistazo. Eso es una decisión de producto: asumes la tensión y la dejas escrita. Lo que no vale es quitarla sin argumento, ni mantenerla fingiendo que nadie te la discutió.

Dejar el alcance escrito en disco

⚠️ **Sal de plan mode antes de este.** Es el **primero que escribe a disco**, y con plan mode puesto el agente planifica pero no crea el archivo. `Shift+Tab` hasta el modo normal, el que pide permiso antes de cada escritura, y confírmalo en la barra.

```
Consolida lo anterior en un alcance final en 5 bloques (problema, usuarios, propuesta de valor, alcance, NO-alcance), listo para ser la base de un PRD. Guarda ese alcance consensuado en docs/prd/alcance-mvp.md, para que no dependa de la conversación. Todavía no escribas el PRD.
```

"Para que no dependa de la conversación" es la línea que hace el trabajo. Todo lo acordado hasta ahora vive en un chat que se va a perder, se va a compactar o se va a cerrar. En cuanto está en un archivo, el prompt siguiente puede releerlo en vez de fiarse de lo que quede en el contexto.

Y **"todavía no escribas el PRD"** evita el atajo servicial: si no se lo dices, te entrega el PRD entero aquí y te saltas el paso de mirar el esqueleto de decisiones antes de vestirlo.

> 🧯 **Abre `docs/prd/alcance-mvp.md` y compruébalo, son quince segundos.** Que la **fecha de vencimiento** esté en el alcance y no en el NO-alcance, y que el **filtrado por estado** también esté dentro. Ese archivo se declara autoritativo a sí mismo, así que corregirlo de palabra no vale: hay que pedirle que **reescriba el archivo**. El prompt siguiente arranca leyéndolo, y lo que no esté ahí no llega al PRD.

Del alcance al PRD

```
Lee @docs/prd/alcance-mvp.md.
Redacta un PRD de MVP para FlowSync en docs/prd/flowsync-mvp.md. Estructura: 1) Problema y contexto, 2) Usuarios y jobs-to-be-done, 3) Propuesta de valor, 4) Alcance / Fuera de alcance, 5) Épicas del MVP: exactamente estas tres y en este orden, E1 «Cuentas y acceso», E2 «Gestión de tareas», E3 «Actividad del equipo» (una línea cada una: nombre + qué agrupa), 6) Requisitos funcionales a nivel producto (numerados, RF-1…; el qué debe hacer el sistema, NO endpoints ni tablas ni modelo de datos), 7) Requisitos no funcionales, 8) Restricciones (stack actual: AdonisJS 7 + React 19; auth ya existe), 9) Métricas de éxito. Sé concreto y testable. No incluyas diseño técnico (ER, arquitectura, diagramas C4): esto es producto. No inventes cifras de mercado; si algo es supuesto, márcalo como [SUPUESTO].
```

Es el prompt más largo de la sesión y cada trozo está por algo.

Empieza leyendo el archivo, no el chat: por eso el paso anterior lo guardó.

Las nueve secciones van enumeradas y las tres épicas van cerradas por nombre y orden. Lo que sale de aquí lo consumen los prompts siguientes por su nombre (`E2 «Gestión de tareas»`), así que si el modelo elige otra descomposición, la segunda mitad de la sesión se queda sin dónde agarrarse.

"NO endpoints ni tablas ni modelo de datos" y **"no incluyas diseño técnico"** son el guardarraíl central. Si lo dejas seguir, el "PRD" crece solo: PRD → modelo de datos → diagramas C4 → casos de uso → diagrama de secuencia. Parece rigor y parece completo, y es la trampa: ese detalle deriva del código y en cuanto tocas una tabla, el diagrama de tu PRD miente.

"No inventes cifras de mercado; si algo es supuesto, márcalo como [SUPUESTO]" saca las alucinaciones a la superficie en vez de dejarlas disfrazadas de dato.

Si aun así se cuela detalle técnico —pasa a veces, pese al prompt—, se sube de nivel en vivo:

```
Revisa el PRD y saca cualquier detalle técnico que se haya colado (nombres de tabla, endpoints, tipos de campo). Súbelo a lenguaje de producto: qué necesita el usuario, no cómo se implementa.
```

Pedirle que ataque su propio PRD

```
Ahora cambia de sombrero: sé un revisor de producto adversarial. Lee el PRD que acabas de escribir y atácalo. ¿Qué requisitos son ambiguos o no testables? ¿Qué supuesto, si es falso, tumba el MVP entero? ¿Dónde hay scope creep escondido? ¿Qué métrica de éxito no se puede medir de verdad? Prioriza por riesgo.
```

Le estás pidiendo que rompa su propio trabajo, y las cuatro preguntas no son intercambiables: **requisitos no testables**, **el supuesto que tumba el MVP entero**, **scope creep escondido** y **métricas que no se pueden medir de verdad**. Cada una caza una familia distinta de defecto. **"Prioriza por riesgo"** es lo que evita que te devuelva quince observaciones planas.

⚠️ **No esperes un resultado concreto.** Los hallazgos **cambian en cada ejecución**: no hay un "hallazgo bueno" que vaya a salir siempre. Lo estable es la forma, no el contenido.

> 💡 **El valor está en ceder cuando tiene razón.** Si nunca cambias nada, esto es teatro. La IA no está para darte la razón; está para que tomes mejores decisiones.

Qué se acepta y qué va a puntos abiertos

Como los hallazgos cambian, aquí no se guioniza "*acepto el número 3*". Se guioniza **la política**, y el agente elige por ti aplicándola.

```
De tus hallazgos acepto uno y el resto van a puntos abiertos.

Acepto el que sea una CONTRADICCIÓN INTERNA del documento: donde el PRD se pide a sí mismo algo que él mismo prohíbe, o promete algo que su propio alcance impide cumplir. Eso es un fallo mío, no una opinión discutible, y se arregla en un párrafo. Aplícalo, y resuélvelo por el lado honesto: si algo no se puede medir o no se puede cumplir, se declara así, sin maquillarlo ni inventarle una mitigación de adorno.

Los que sean decisiones de producto —qué se ve, en qué orden, con qué lente, cuánto de algo— no se deciden en treinta segundos delante de una clase. Pásalos a la sección de puntos abiertos, cada uno con tu argumento resumido y con lo que haría falta para decidirlo.

Los que tú mismo marcas como subsanables durante la construcción, a puntos abiertos también.

Dos cosas NO se tocan, aunque las hayas cuestionado bien: la fecha de vencimiento y el filtrado por estado se quedan DENTRO del alcance y bajo la épica E2. Si algún hallazgo tuyo los ataca, va a puntos abiertos, nunca al NO-alcance.

Actualiza docs/prd/flowsync-mvp.md con eso y nada más. No reescribas secciones que ningún hallazgo toque.
```

Cuatro cosas que hacen este prompt, y conviene entenderlas porque es el patrón más reutilizable de la sesión:

Se acepta la contradicción interna y solo esa. Una contradicción no se discute: el documento se delata solo, el arreglo cabe en un párrafo y no reabre el alcance. Las decisiones de producto, en cambio, se decidirían mal con el reloj corriendo.

"Resuélvelo por el lado honesto" impide la salida fácil: si algo no se puede medir, se declara que no se puede medir, en vez de inventarle una mitigación de adorno.

El blindaje explícito de la fecha de vencimiento y el filtrado por estado. Son las dos historias que se descomponen después; si un hallazgo las ataca, va a puntos abiertos, nunca al NO-alcance.

"No reescribas secciones que ningún hallazgo toque" evita el efecto colateral más caro de todos: que al aplicar dos correcciones te reescriba el documento entero y pierdas lo que ya estaba bien.

Versionar el PRD

Dos líneas, en este orden, dentro de `claude`:

```
!git add docs/
```

```
/commit
```

Van **siempre juntas**, y no es un detalle: la skill `/commit` que montaste en el Módulo 1 commitea lo que ya está en *staging* y **no hace `add` por su cuenta**. El PRD es un archivo nuevo sin trackear, así que sin el `add` el staging está vacío y la skill no tiene nada que commitear.

El mensaje lo redacta la propia skill. Debería salir esto; si sale otro, pídeselo:

```
docs(prd): PRD del MVP de FlowSync
```

> 💡 **Esto es lo que separa un PRD vivo de un Word muerto.** Ahora vive en Git: cambia con historial, se revisa en un PR, y el agente lo va a leer en el paso siguiente para sacar las historias. "Fuente de verdad viva" no es un concepto, es un archivo commiteado.

Si quieres, aprovecha para mirar `/context` y ver cuánto contexto llevas gastado (`/compact` si va largo).

Partir la épica en historias

```
Lee @docs/prd/flowsync-mvp.md.
Descompón la épica E2 "Gestión de tareas" en historias de usuario.
Cada historia en formato "Como <rol>, quiero <acción> para <beneficio>".
Aplica INVEST: pequeñas, testables. NO escribas criterios todavía, solo el listado.
Valida cada historia contra el alcance del PRD: si alguna se sale del MVP, márcala
explícitamente como "fuera de alcance MVP" en vez de colarla.
```

"NO escribas criterios todavía, solo el listado" es deliberado: primero el mapa, después el detalle. Si le dejas escribir criterios aquí, te devuelve un muro de texto en el que no se ve la forma de la épica.

Y la última línea es la que más rinde: **que ella misma marque lo que se sale del MVP** en vez de colarlo. Es la frontera que fijaste en el PRD, comprobándose sola. Si no marca ninguna, no lo des por bueno: pregúntale cuál no cabe y por qué.

> 📌 **Los identificadores `FS-118` y `FS-142` son convención nuestra, no algo que salga de la IA ni de ninguna herramienta.** Cuando las historias suban al tablero, Jira les pondrá su propia clave; nuestro identificador vive en el título y en la etiqueta.

Criterios como regla de negocio, no como spec técnica

Es el corazón de la segunda mitad.

```
De las historias que acabas de listar, coge la de la fecha de vencimiento y las tareas vencidas. Si te salieron separadas (poner o quitar la fecha por un lado, ver si se ha pasado de plazo por otro), consolídalas en UNA sola historia antes de seguir: para nosotros es una única historia y se llama FS-118.
Escribe sus criterios de aceptación en formato Given/When/Then, redactados en español (DADO / CUANDO / ENTONCES).
Reglas MUY importantes:
- Un criterio de historia es una REGLA DE NEGOCIO observable: qué es verdad para el usuario. NO uses endpoints, status codes ni nombres de campo internos (nada de "PATCH /tasks/:id", "422", "isOverdue"). Ese detalle técnico es de la fase de implementación, no de aquí.
- Incluye el camino feliz Y los edge cases / errores que se me puedan olvidar.
- Marca los criterios que propongas tú para mi revisión.
```

La consolidación del principio existe porque **la descomposición no devuelve tus historias con tus nombres**: en las pruebas la IA partió esta en dos (poner la fecha por un lado, ver el vencimiento por otro). Por eso el prompt la pide **por contenido** y no por su nombre, y le fija el nuestro.

La restricción central es **"un criterio de historia es una REGLA DE NEGOCIO observable"**, con los ejemplos de lo prohibido dentro (`PATCH /tasks/:id`, `422`, `isOverdue`). Sin esos ejemplos concretos la prohibición es abstracta y se la salta. Un criterio de historia dice "*una tarea está vencida solo si su fecha ya pasó y no está terminada*": una verdad sobre el producto que entiende cualquiera del equipo. El endpoint, el código de error y el nombre del campo son la misma regla ganando precisión al bajar de nivel, y bajan después.

"Los edge cases que se me puedan olvidar" es lo que hace útil el ejercicio. Suelen salir: una tarea puede tener fecha o no; está vencida solo si la fecha pasó **y** no está terminada; una tarea terminada nunca aparece vencida; se puede poner una fecha en el pasado.

"Marca los criterios que propongas tú" deja visible qué es tuyo y qué es suyo, que es lo que te permite revisar en vez de aceptar en bloque.

> 🔍 **No todo lo que propone es correcto, y ese es el ejercicio.** Suele proponer **rechazar** las fechas de vencimiento en el pasado. Es discutible: al importar tareas viejas puede interesarte permitirlas y que salgan vencidas sin más. La IA propone, tú decides, y el criterio lo firmas tú.

La historia ligera: filtrar por estado

Misma receta, mucho más rápida.

```
Ahora la historia de filtrar las tareas por estado para centrarse en lo pendiente. Si no apareció al descomponer E2 porque ese requisito quedó bajo otra épica, tómala igualmente: para nosotros es una historia de E2 y se llama FS-142.
Mismos criterios: reglas de negocio observables en Given/When/Then, en español (DADO / CUANDO / ENTONCES), sin endpoints ni status codes. Incluye el caso de que se pida un estado que no existe (el sistema debe avisar del error, no devolver una lista vacía en silencio).
```

La primera frase es una red: si el filtrado por estado quedó bajo otra épica en el PRD, el prompt la coge igualmente y la adjudica donde toca, en vez de dejarte parado.

Y fíjate en el criterio del error: **"que avise, no que devuelva una lista vacía en silencio"**. A nivel de historia se dice así de simple. Que ese aviso acabe siendo un `422` con un enum validado es cosa de la implementación.

Sacar las historias del chat al repositorio

Hasta aquí solo viven en la conversación.

```
Guarda en docs/backlog/E2-gestion-tareas/ SOLO las dos historias que acabamos de enriquecer con criterios de aceptación, un archivo markdown por historia. Las demás historias del listado no se guardan todavía. La de fechas de vencimiento va en us-fechas-vencimiento.md. Cada archivo: el título de la historia, su identificador, su formulación "Como <rol>, quiero <acción> para <beneficio>" y sus criterios de aceptación en Given/When/Then, en español (DADO / CUANDO / ENTONCES), tal como los hemos aprobado y con mis ediciones incluidas. Sin endpoints, status codes ni nombres de campo internos. Los identificadores los fijo yo, no los inventes: la historia de fechas de vencimiento es FS-118 y la de filtrar por estado es FS-142.
```

Dos restricciones que evitan un fallo tonto y caro. **"SOLO las dos historias"**: si guarda las diez, el repositorio se llena de historias sin criterios que nadie ha revisado. Y **el nombre de archivo fijado a mano** (`us-fechas-vencimiento.md`), porque el prompt siguiente lo abre por su nombre: si el modelo elige el suyo, el paso siguiente no encuentra nada.

Lo mismo con **"los identificadores los fijo yo, no los inventes"**. Es la regla general que te llevas de aquí: **lo que un paso posterior consume literalmente, se fija en el prompt que lo crea.**

Y se versiona igual que antes:

```
!git add docs/
```

```
/commit
```

```
docs(backlog): historias de E2 con criterios de aceptación
```

De historia a tickets

```
Coge @docs/backlog/E2-gestion-tareas/us-fechas-vencimiento.md.
Descompón esta historia en TICKETS: cada ticket = una unidad de trabajo que una persona termina en una sesión (media jornada máx).
Para cada ticket: ID derivado de la historia (la historia es FS-118, así que sus tickets son FS-118.1, FS-118.2, FS-118.3…), título, tipo (uno de estos: Endpoint/API, Migración/DB, Modelo/Dominio, Frontend, Bug o Test), Definition of Done según su tipo, y dependencias (qué ticket lo bloquea).
El ticket HEREDA los criterios de la historia; su Definition of Done es una checklist de "cómo lo entregamos" (tests, manejo de error, convenciones), NO criterios nuevos ni estimación en horas.
El ticket NOMBRA la capa que toca (migración, modelo, endpoint, UI), pero NO diseña: nada de tipos de columna, si admiten nulos, índices, nombres de ruta ni status codes. Esa decisión es de la implementación.
```

Aquí cambia la altitud, y conviene verlo: la historia dice la **regla**; el ticket dice **qué capa** hay que tocar para cumplirla.

"Una unidad de trabajo que una persona termina en una sesión" es la definición operativa de la granularidad. Sin ella los tickets salen del tamaño que le apetezca.

"El ticket HEREDA los criterios de la historia… NO criterios nuevos ni estimación en horas" corta de raíz el antipatrón: un ticket que inventa criterios propios rompe la trazabilidad con la historia, y un ticket con horas dentro finge una precisión que nadie tiene.

"NOMBRA la capa que toca, pero NO diseña" es la frontera fina. Un ticket puede decir "migración" y "endpoint"; lo que no puede decir es de qué tipo es la columna, si admite nulos, qué índice lleva ni qué status code devuelve. Eso se decide al implementar.

Los tickets de la historia ligera

```
Ahora coge la historia FS-142 "Filtrar tareas por estado", de
@docs/backlog/E2-gestion-tareas/.
Descompón esta historia en TICKETS con la misma convención que la anterior: ID derivado
de la historia (la historia es FS-142, así que sus tickets son FS-142.1, FS-142.2…),
título, tipo (uno de estos: Endpoint/API, Migración/DB, Modelo/Dominio, Frontend, Bug o
Test), Definition of Done según su tipo, y dependencias (qué ticket lo bloquea).
El ticket HEREDA los criterios de la historia; su Definition of Done es una checklist de
"cómo lo entregamos" (tests, manejo de error, convenciones), NO criterios nuevos ni
estimación en horas.
El ticket NOMBRA la capa que toca (migración, modelo, endpoint, UI), pero NO diseña:
nada de tipos de columna, si admiten nulos, índices, nombres de ruta ni status codes.
Esa decisión es de la implementación.
```

Misma convención, historia mucho más pequeña: suele bastar un ticket.

El grafo de dependencias

```
Muéstrame las dependencias entre los tickets de FS-118 como un grafo simple (qué bloquea
a qué) y el orden de implementación recomendado.
```

Pide **dos cosas a la vez**: el grafo y el orden recomendado. El grafo solo es decoración; el orden es lo accionable. Lo típico es que la migración bloquee al modelo, el modelo al endpoint, la UI cuelgue del endpoint y los tests vayan al final.

Las dependencias mal puestas son una de las causas más comunes de trabajo bloqueado, y por eso se ordenan **antes** de estimar.

Estimar con las tallas, y con sus sesgos

```
Estima cada ticket en t-shirt sizing (S/M/L) y da una nota de riesgo. No inventes horas
exactas; explica en qué se basa cada talla.
```

"No inventes horas exactas; explica en qué se basa cada talla" es todo el prompt. Lo primero evita la falsa precisión; lo segundo te da algo que discutir en vez de un número que aceptar.

⚠️ **La estimación asistida arrastra tres sesgos, y conviene nombrarlos:**

- **Optimismo por contagio**: si te descompone la épica en treinta segundos, tu cabeza calibra contra lo rápido que se generó, no contra lo que cuesta hacerlo.
- **Falsa precisión**: te devuelve "3,5 puntos" y ese decimal es ficticio. Redondea siempre a tu escala.
- **Homogeneización**: si se lo pides todo de golpe, las tallas salen con la misma forma y se enmascara lo que esconde complejidad real.

Y encima **no conoce la velocidad real de tu equipo**. No hay multiplicador mágico. La IA rompe la página en blanco; la estimación la firma el equipo.

> 💡 **Sobre la unidad**: aquí se usa t-shirt sizing porque cabe en una demo y porque lo que interesa ver es la conversación, no la técnica. En un sprint planning real la t-shirt se queda gruesa: ahí sirve Fibonacci, y las tallas se quedan arriba, para el roadmap.

Priorizar: impacto, complejidad… y dependencias

```
Coloca en una matriz impacto vs complejidad las historias de E2 "Gestión de tareas" (incluidas las que quedaron fuera del MVP) y también el sync en tiempo real de E3 "Actividad del equipo", y propón un orden de backlog. Marca los "quick wins" (alto impacto, baja complejidad).
```

Fíjate en que el prompt incluye a propósito **las historias que quedaron fuera del MVP** y una de otra épica. Al NO-alcance se va **con fecha, no al olvido**: colocar lo de fuera es lo que te dice qué viene después y por qué no es ahora.

> 🚨 **Y aquí está la trampa preciosa de este bloque.** La matriz va a marcar como *quick wins* las dos historias del día. Pero la primera del backlog **no es ninguna de las dos**: es "crear una tarea". ¿Por qué? Porque la matriz ordena por **valor**, y un backlog real se ordena por valor **y dependencia**. No se puede filtrar por estado una lista de tareas que todavía nadie puede crear. **Un quick win que aún no puedes construir no es lo primero.** Eso es exactamente lo que separa un backlog bonito de un backlog ejecutable.
>
> Si la IA coloca una de las dos en el puesto 1, pídele que revise las dependencias: es un fallo suyo y se ve mejor corrigiéndolo.

Dejar el backlog escrito antes de llevarlo al tablero

```
Escribe en docs/backlog/ el backlog que acabamos de construir: los tickets de cada historia (ID derivado FS-118.1…, título, tipo, Definition of Done y dependencias), el grafo de dependencias y el orden de implementación, y la matriz impacto/complejidad de las historias de E2 con el orden de backlog priorizado. Respeta lo que hemos decidido en vivo; no añadas horas ni criterios nuevos.
```

"Respeta lo que hemos decidido en vivo; no añadas horas ni criterios nuevos" es necesario porque a estas alturas la conversación es larga y el modelo tiende a "completar" lo que le parece que falta.

> 📌 **Las tallas y la nota de riesgo se quedan en pantalla a propósito**, no bajan a `docs/backlog/` ni suben al tablero. No es un olvido: es la consecuencia de lo que acabas de leer sobre los sesgos. Se versiona el contrato, no una cifra que el equipo aún no ha calibrado.

Materializar el backlog en Jira

```
Materializa el backlog en el tablero FLOW de Jira usando el MCP de Atlassian. Hazlo
SECUENCIAL, en este orden, sin lanzar creaciones en paralelo:

1. Crea la HISTORIA FS-118 como issue de tipo "Historia". Título: «FS-118 — Fechas de
vencimiento y tareas vencidas». Descripción: sus criterios de aceptación en
Given/When/Then, en español (DADO / CUANDO / ENTONCES), tal como los aprobamos (reglas
de negocio, sin endpoints ni status codes). Etiquetas: "E2" y "FS-118".
2. Lee la clave que Jira devuelve para esa historia (tiene la forma FLOW-nnn) y úsala en
el paso 3.
3. Crea cada ticket de esa historia (FS-118.1, FS-118.2, FS-118.3…) como issue de tipo
"Subtarea", pasando el parámetro parent con la clave FLOW-nnn de la historia. En cada
subtarea: título con su ID derivado («FS-118.1 — …»), descripción con su tipo y su
Definition of Done, etiquetas "E2" y su ID.
4. Repite los pasos 1-3 con la HISTORIA FS-142 «Filtrar tareas por estado» y los tickets
que le hayan salido (FS-142.1, FS-142.2…, los que sean). Si esa historia no tiene ningún
ticket, créala igualmente como "Historia" y no le cuelgues subtareas.
5. Crea por último la HISTORIA que encabeza el orden priorizado, como issue de tipo
"Historia" y SIN subtareas. Título: «E2-1 — Crear tarea con solo el título».
Descripción: que es la primera del orden de backlog por ser prerrequisito de todo lo
demás; que junto con E2-2, E2-3 y E2-4 forma la base de la capability de tareas
(requisitos RF-5 a RF-9 del PRD: crear con solo el título, título obligatorio,
responsable por defecto, los tres estados fijos y cambiar el estado desde la lista); y
que sus criterios de aceptación viven en el backlog del repositorio, en docs/backlog/,
no en este tablero: hoy en directo hemos enriquecido solo las dos historias de arriba.
Etiquetas: "E2" y "E2-1".

Restricciones que debes respetar:
- Los tipos de issue de este proyecto están EN ESPAÑOL: usa literalmente "Historia" y
"Subtarea". "Story" o "Sub-task" no resuelven.
- Una subtarea NO se puede crear sin parent: la historia va siempre primero y su clave
se encadena a cada subtarea.
- No inventes claves de Jira: la clave real la asigna Jira (FLOW-nnn). FS-118 y FS-118.1
son nuestra convención y viven en el título y las etiquetas, nunca como clave.
```

Es el prompt más frágil de la sesión, y sus restricciones son casi todas cicatrices:

"Hazlo SECUENCIAL" porque una subtarea **no se puede crear sin su `parent`**, y el `parent` es una clave que Jira todavía no ha asignado. Lanzarlo en paralelo falla, y falla a medias, dejando issues sueltas en un tablero compartido.

"Lee la clave que Jira devuelve" hace explícito ese encadenado, en vez de confiar en que lo deduzca.

"Los tipos de issue están EN ESPAÑOL" porque `Historia` y `Subtarea` son literales de esa instancia. `Story` y `Sub-task` no resuelven, y el fallo llega cuando ya has creado la mitad.

"No inventes claves de Jira" separa dos numeraciones que es fácil confundir: la clave real la asigna Jira (`FLOW-nnn`); `FS-118` es nuestra convención y vive en el título y en las etiquetas.

Y el punto 4 contempla **los dos casos** —que la historia ligera tenga tickets o que no tenga ninguno—, para que el prompt se pegue igual sin editarlo en vivo.

> 🧹 **Si lo corres sobre un tablero compartido, límpialo al terminar.** El MCP de Atlassian **no borra ni archiva**: solo crea, edita, transiciona y comenta. Se hace a mano desde la interfaz de Jira. Borrar la historia padre se lleva sus subtareas por delante.

Versionar el backlog

Igual que las dos veces anteriores:

```
!git add docs/
```

```
/commit
```

```
docs(backlog): tickets de E2 priorizados
```

Lo que te llevas

- Un **alcance consensuado** en disco (`docs/prd/alcance-mvp.md`), que dejó de depender de una conversación.
- Un **PRD de producto** versionado (`docs/prd/flowsync-mvp.md`), sin una tabla ni un endpoint dentro, con sus supuestos marcados y sus puntos abiertos declarados.
- Una **épica descompuesta en historias**, con las que se salen del MVP marcadas por la propia IA.
- **Dos historias con criterios de aceptación** escritos como reglas de negocio observables, no como specs técnicas.
- Esas historias **partidas en tickets** con su Definition of Done, su grafo de dependencias y su orden.
- Todo **materializado en un tablero** y **versionado en el repositorio**, sin haber escrito una línea de la feature.

> **Y la frase para llevarse del día:** el criterio de aceptación que escribes hoy es el input directo de la implementación de mañana. Por eso el backlog vive junto al código y no en un documento que nadie vuelve a abrir.

> ⚠️ **Si algo no te sale igual, no es que lo hayas hecho mal.** Estos prompts se ejecutaron muchas veces al preparar la sesión y **el resultado nunca fue idéntico dos veces**: cambia el número de historias, cambian los hallazgos del revisor adversarial, cambia dónde coloca cada cosa. Lo que se repite es **la forma**; el número exacto y la redacción, no.